

VDR-LLM-Prolog on Dedicated Silicon

From FPGA Proof-of-Concept to Integer-Native GPU Architecture

Registry: [?]

Series Path: [HOWL-VDR-1-2026] → [HOWL-VDR-2-2026] → [HOWL-MATH-3-2026] → [HOWL-MATH-4-2026] → ... → [HOWL-VDR-14-2026] → ... → [HOWL-VDR-21-2026] → [?]

DOI: 10.5281/zenodo.zzz

Date: May 2026

Domain: Computer Architecture / ASIC Design

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic's Opus 4.6.

Abstract

The VDR-LLM-Prolog FPGA implementation [HOWL-VDR-21-2026] validates an architectural principle: Q335 exact integer arithmetic — where every value is a 384-bit numerator over an implicit fixed denominator of 2^{335} , and division by that denominator is bit extraction requiring zero logic — maps naturally to parallel hardware. The FPGA achieves this at 150 MHz on 10 custom cores in a \$200 system-on-chip. This paper asks what happens when that architecture moves to dedicated silicon designed for it. Modern GPU fabrication at 4-5nm provides transistor budgets exceeding 80 billion, clock speeds of 2-2.5 GHz, and memory bandwidths of 3-5 TB/s via HBM3. Current GPUs dedicate substantial die area to floating-point units, tensor cores with float accumulation, and special function units for transcendentals (sin, cos, exp, rsqrt) — none of which VDR-LLM-Prolog uses. This paper specifies an integer-native processor that reclaims that area for wide integer arithmetic: 384-bit ALUs with 1-2 cycle multiply (versus 9 cycles on FPGA), SHR335 as a routing decision (zero gates, zero power, zero latency beyond wire delay — the same property that makes it zero logic on FPGA, now at thousands of units), and a reduction network that produces exact results at every level. The design targets 5,120 Q335 cores organized into 80 streaming multiprocessors, projecting approximately 5 trillion Q335 multiplications per second — sufficient that the arithmetic is memory-bandwidth-bound, not compute-bound, on workloads where VDR-18 showed total multiply counts of thousands to millions per prompt. The paper specifies the core microarchitecture, the memory hierarchy, the programming model, the die area estimates, and the performance projections, treating the FPGA's validated ISA principles as the architectural contract and modern GPU fabrication as the implementation technology.

1. What the FPGA Proved

The FPGA implementation on Zynq-7020 [HOWL-VDR-21-2026] validated five architectural principles that are independent of fabrication technology.

First, Q335 divmod is free in hardware at any clock speed. The SHR335 instruction extracts bits [767:335] as the quotient and bits [334:0] as the remainder. On the FPGA this is fixed wiring — zero lookup tables, zero flip-flops, zero switching power. On dedicated silicon it is the same: bit positions are named differently on the output bus. This is not an optimization. It is a consequence of the mathematical decision to fix the denominator at a power of two. The decision was made for arithmetic reasons (controlled denominator growth). It produces a hardware benefit (free division) as a structural consequence. This benefit scales linearly with core count: 10 cores get 10 free divmods per multiply cycle; 5,120 cores get 5,120.

Second, Q335 arithmetic is perfectly uniform. Every value is the same width (384 bits). Every addition is the same operation (1 cycle). Every multiplication is the same operation (9 cycles on FPGA, fewer with dedicated logic). There are no special cases, no data-dependent branches, no variable-width operands. This is the workload profile that massively parallel hardware achieves peak utilization on. VDR-18 demonstrated this empirically: surrogate softmax achieved 80-95% GPU utilization versus 40-60% for conventional softmax, specifically because the integer path has no transcendental divergence causing warp divergence.

Third, all VDR operations compose from a small set of integer primitives. The FPGA's 53-instruction ISA covers the complete VDR arithmetic, remainder tree management, Prolog matching, and batch processing requirements. The instructions are regular: fixed 32-bit encoding, four formats, deterministic cycle counts. No instruction requires floating-point hardware, transcendental evaluation, or variable-latency execution.

Fourth, the division between embarrassingly parallel operations (fact matching, batch arithmetic, constraint evaluation) and sequential operations (Prolog backtracking, functional remainder resolution, KB tree management) is clean. The FPGA validates that the parallel operations constitute the vast majority of computation time, while the sequential operations are control-plane decisions that a scalar host processor handles efficiently.

Fifth, bit-identical results across implementations. The FPGA produces results identical to the Python reference implementation for every test in the 884-test validation suite. The arithmetic is deterministic integer computation. The same property holds regardless of fabrication technology: the same integer inputs processed by the same integer operations produce the same integer outputs on any implementation.

These five principles define the contract for dedicated silicon. The implementation technology changes. The architectural contract does not.

2. What Current GPUs Waste on VDR Workloads

A modern GPU at the performance tier of NVIDIA H100 dedicates significant die area to computational units that VDR-LLM-Prolog does not use.

2.1 Floating-Point Units

Each streaming multiprocessor contains FP32 units (typically 64-128 per SM), FP64 units (typically 32 per SM on compute-oriented SKUs), and FP16/BF16 units (often shared with or double-pumped from FP32 paths). These implement IEEE 754 arithmetic with rounding modes, denormalized number handling, NaN propagation, and sign-magnitude representation. VDR-LLM-Prolog performs zero floating-point operations. Every value is an integer. Every computation is integer addition, multiplication, comparison, or bit manipulation. The entire floating-point datapath — mantissa alignment, exponent comparison, rounding logic, special-value handling — is unused silicon.

2.2 Tensor Cores

Fourth-generation tensor cores perform matrix multiply-accumulate on tiles of FP16, BF16, FP8, or INT8 operands with FP32 accumulation. The accumulation is floating-point even when the inputs are integer. The tile sizes (typically $16 \times 16 \times 16$ or $8 \times 8 \times 16$) are optimized for the dimensions of neural network layers at float precision. VDR-LLM-Prolog’s matrix operations use 384-bit integer operands with exact integer accumulation. The tile geometry, the accumulation datapath, and the input precisions of current tensor cores do not match VDR requirements.

2.3 Special Function Units

Each SM contains special function units (SFUs) that compute transcendental functions: sine, cosine, exponential, reciprocal square root, logarithm. These are hardware function approximators — typically using polynomial approximation or lookup-table-interpolation methods — that produce results accurate to float precision. VDR-LLM-Prolog computes transcendentals through functional remainders: Newton iteration, Taylor series, and Borwein acceleration evaluated as exact rational arithmetic. The system never invokes a hardware transcendental approximation. SFUs are unused silicon.

2.4 The Opportunity

Removing float units, float tensor cores, and SFUs from each SM frees transistor budget for wide integer ALUs. The precise area savings depend on the specific microarchitecture, but published die analyses of modern GPUs indicate that floating-point datapaths (including tensor cores) consume 40-60% of each SM’s logic area. Reclaiming that area for 384-bit integer units yields a proportional increase in integer compute density per die.

3. Q335 Core Microarchitecture

3.1 The Q335 Integer Unit

The fundamental compute element is a Q335 integer unit (QIU): a 384-bit ALU with dedicated support for Q335 arithmetic, remainder management, and Prolog term matching.

The QIU contains a 384-bit carry-select adder (1 cycle, same structure as the FPGA design), a dedicated 384×384 -bit multiplier producing a 768-bit result in 1-2 cycles (full parallel multiplier array, not time-multiplexed as on FPGA), a 384-bit barrel shifter with SHR335 as a zero-gate routing path, a 384-bit comparator with early termination, and a cross-multiply unit for VDR fraction unification (2-3 cycles with the faster multiplier, versus 19 on FPGA).

The critical difference from the FPGA implementation: the multiplier. The FPGA uses 5 DSP48E1 slices time-multiplexed across 9 cycles because DSP slices are a scarce resource on the Zynq-7020 (220 total, 50 used). Dedicated silicon has no such constraint. A full 384×384 parallel multiplier array — implemented as a tree of 64-bit multipliers with carry-save accumulation — produces the 768-bit result in 1-2 pipeline stages. At 2 GHz, each Q335 multiply takes 0.5-1.0 nanoseconds. The SHR335 extraction of quotient and remainder adds zero additional latency: it is a routing decision in the physical layout, the same as on FPGA but at $13 \times$ the clock frequency.

3.2 Warp Organization

Following the established GPU programming model, 32 QIUs execute in lockstep as a warp. Each QIU in the warp processes a different data element using the same instruction. This maps directly to VDR workloads: a warp of 32 QIUs computes 32 Q335 additions in one cycle, 32 Q335 multiplications in 1-2 cycles, or evaluates 32 Prolog fact matches simultaneously.

Warp-level uniformity is guaranteed by the nature of Q335 arithmetic. Every QIU executes the same instruction for the same number of cycles on every data element. There is no warp divergence because there are no data-dependent branches in the arithmetic path. The JCLOSED conditional (testing whether a remainder is zero) is the only potential divergence point, and the expected active spill rate of below 1% means fewer than 1 in 100 elements takes the nesting path. Predicated execution handles this without warp splitting.

3.3 Streaming Multiprocessor

Each SM contains 64 QIUs organized as 2 warps. The SM also contains a local reduction unit (binary tree of 384-bit adders for intra-SM reduction), a shared BRAM region for Q335 constants, predicate lookup tables, and per-SM configuration, a register file providing 16×384 -bit registers per QIU (versus 8 on FPGA — the larger count reduces register pressure in complex microprograms), a remainder tree BRAM of 4KB per QIU (32 nodes versus 16 on FPGA), an instruction cache (replacing the FPGA's per-core instruction BRAM), and a warp scheduler managing instruction issue.

The SM operates at 2 GHz. Each warp of 32 QIUs issues one instruction per cycle. With

2 warps per SM, the SM can issue 2 instructions per cycle (one per warp), sustaining 64 QIU operations per cycle when both warps have independent work.

3.4 Global Architecture

The full chip contains 80 SMs with 64 QIUs each, totaling 5,120 QIUs. This is comparable to the integer ALU count of current high-end GPUs (H100 has 16,896 FP32 cores across 132 SMs) but each QIU is wider (384-bit versus 32-bit) and specialized for exact arithmetic.

A global reduction network connects all 80 SMs. The network is a 7-level tree of 384-bit adders ($\log_2(80) \approx 7$, rounded up with fan-in considerations). Each level takes 1 cycle. A global REDUCE operation completes in 7 cycles — delivering an exact 384-bit sum across all 5,120 QIUs in 3.5 nanoseconds at 2 GHz. This supports softmax surrogate denominators, global norms, and aggregation operations that span the entire chip.

4. The SHR335 at Scale

The FPGA paper established that SHR335 is zero logic: fixed wiring extracting bit positions from a register. On dedicated silicon this property is worth examining at scale because it compounds.

Each QIU performs Q335 multiplication followed by SHR335 extraction. The multiply produces a 768-bit result. SHR335 routes bits [767:335] to the quotient output and bits [334:0] to the remainder output. In the physical layout this means 768 wires from the multiplier output are split into two groups by their position — no gates, no transistors, no switching energy, no propagation delay beyond wire routing. The quotient and remainder are available at the multiplier output simultaneously with the product itself.

With 5,120 QIUs, each potentially executing a Q335 multiply every 1-2 cycles at 2 GHz, the chip performs 2.5-5.0 trillion multiplications per second. Each multiplication includes a free divmod. That is 2.5-5.0 trillion exact integer divisions per second at zero computational cost, zero power consumption (beyond the wires), and zero additional latency. The division that is the computational heart of VDR arithmetic — the operation that makes the fixed-frame scheme work — is performed 5 trillion times per second by the physical layout of metal interconnect.

No other arithmetic system has this property. Floating-point division is a multi-cycle operation requiring iterative convergence or lookup-table interpolation. Arbitrary-precision rational division requires multi-word quotient computation. Even integer division by non-power-of-two denominators requires dedicated divider circuits. The Q335 design decision to fix the denominator at 2^{335} — a mathematical choice — yields a hardware property — free division — that scales without limit.

5. Memory Hierarchy

5.1 Register File

Each QIU has 16×384 -bit value registers and 16×384 -bit remainder registers — 12,288 bits per QIU, approximately 1.5 KB. Across 5,120 QIUs: 7.68 MB of register file. This is comparable to the total register file size of current high-end GPUs (H100 has approximately 20 MB of register file across all SMs) despite the wider per-register width, because VDR requires fewer registers per thread — the working set of a Q335 operation is 2-3 source values and 1-2 destination values.

5.2 Remainder BRAM

Each QIU has 4 KB of dedicated SRAM for remainder tree storage: 32 nodes at 128 bytes each. Across 5,120 QIUs: 20 MB. This is local to each QIU and accessed in 1-2 cycles. The remainder BRAM serves the same function as the FPGA’s per-core remainder BRAM but with double the node capacity, supporting deeper nesting without spilling to shared memory.

5.3 Shared Memory per SM

Each SM has 256 KB of shared SRAM, accessible by all 64 QIUs within the SM. This holds the Q335 constant table (22 constants $\times$ 48 bytes = 1,056 bytes, replicated per SM for zero-contention access), predicate lookup tables for Prolog fact matching, tile-local portions of attention matrices during matrix operations, and intermediate reduction results.

Total shared memory: 80 SMs $\times$ 256 KB = 20 MB.

5.4 L2 Cache

A global L2 cache of 96 MB serves as the intermediary between SMs and HBM. This is sized to hold the working set of a VDR-LLM-Prolog prompt: attention matrices for moderate sequence lengths, active KB fact partitions, and gradient buffers during training. VDR-18 identified that most VDR data structures fit in GPU L2 cache, and this property is preserved with the larger L2.

5.5 HBM3 Memory

Six HBM3 stacks provide 96 GB of capacity at 4.9 TB/s aggregate bandwidth. The memory is organized into 6 independently addressable partitions, enabling:

The KB fact store occupies partitions sized to the deployment — from megabytes for small knowledge bases to tens of gigabytes for enterprise deployments with millions of facts.

Model parameters for exact VDR training occupy 384 bits per parameter. A 1-billion parameter model requires 48 GB — fitting within the 96 GB capacity but not leaving much room. This confirms VDR-4’s honest assessment that exact-fraction training is not

practical at production scale on any single device. The target model size for exact training is tens of millions of parameters — fully within the HBM capacity with room for gradients, attention matrices, and KB storage.

For inference (forward pass only, no gradient storage), larger models are feasible. A 7-billion parameter model at 384 bits per parameter requires 336 GB, exceeding single-device HBM capacity but achievable with model parallelism across 4 devices.

6. Programming Model

6.1 Compatibility with VDR-18

The programming model directly implements the five-stream GPU architecture specified in VDR-18. The streams map to the integer-native hardware as follows.

Stream 0 (LLM forward/decode) uses QIU warps for exact matrix multiplications in attention and feedforward layers. Each SM computes rows of the attention matrix or tiles of the feedforward computation. The tensor-core-equivalent operation is 384-bit integer matrix multiply-accumulate with exact accumulation — no rounding at any stage.

Stream 1 (KB query/scan) uses QIU warps for parallel Prolog fact matching. The predicate-major columnar storage from VDR-18 maps directly: all facts sharing a predicate are stored contiguously in HBM, loaded in coalesced bursts to shared memory, and scanned by warps executing the BMATCH instruction equivalent. Scope filtering uses the same bitset mechanism — each QIU checks a candidate fact’s KB ID with one bit-test against the user’s visible-KB bitset.

Stream 2 (VDR primitives) uses QIU warps for batch arithmetic — parameter updates, aggregations, denominator budget checks. These are the embarrassingly parallel operations that fill warps perfectly.

Stream 3 (grammar mask/prep) uses QIU integer comparison for grammar-constrained vocabulary filtering. When a grammar slot accepts one of 4 categorical values, the mask operation zeros all but 4 entries in the logit vector — a parallel integer comparison across the vocabulary.

Stream 4 (provenance compaction) uses DMA engines (unchanged from conventional GPU architecture) for append-only event logging.

The five streams overlap because they access different memory regions and different functional units, exactly as specified in VDR-18. The integer-native hardware eliminates the resource conflict that would occur if streams 0 and 1 competed for the same float units — there are no float units. All streams use integer units, but they access different memory partitions and operate on different data.

6.2 Kernel Launch Model

The host CPU launches kernels on the integer-native GPU using the same grid/block/thread abstraction as conventional CUDA. A Q335 addition kernel launches with block size 64 (matching QIUs per SM) and grid size equal to the number of elements divided by 64. Each thread computes one 384-bit addition. The kernel code uses Q335-specific intrinsics:

The intrinsic `q335_add(a, b)` maps to the WADD instruction equivalent. The intrinsic `q335_mul(a, b)` maps to WMUL followed by SHR335 (the SHR335 is implicit — the hardware always produces quotient and remainder). The intrinsic `q335_reduce_add(value)` maps to the hierarchical reduction network. The intrinsic `prolog_match(predicate_id, args)` maps to the BMATCH sequence.

These intrinsics are the software interface to the hardware operations validated on the FPGA. The calling convention, argument types, and return types are identical. The performance characteristics differ (1-2 cycle multiply versus 9, for example) but the functional semantics are identical, ensuring bit-identical results across FPGA and ASIC implementations.

6.3 The Zig Runtime Interface

The VDR-LLM-Prolog Zig runtime on the host CPU dispatches to the integer-native GPU through the same IOSE-contracted primitive interface used for the FPGA and software paths. The runtime selects the implementation path based on available hardware:

Software path: all operations executed in Zig on the host CPU. FPGA path: parallel operations dispatched to the FPGA via AXI registers and DMA. GPU path: parallel operations launched as kernels on the integer-native GPU.

The selection is transparent to the caller. A `kb_query` builtin (B378) produces the same result regardless of whether the fact scan runs on a Zig hash map, an FPGA BMATCH sequence, or a GPU warp executing integer comparisons. The IOSE declaration is the contract. The implementation is the acceleration.

7. Die Area Estimates

7.1 Per-QIU Area

Component	Transistor Estimate	Area at 4nm (mm ²)	Notes
384-bit multiplier (full parallel)	~2.5M	0.020	Parallel 64-bit multiplier tree with carry-save

Component	Transistor Estimate	Area at 4nm (mm ²)	Notes
384-bit adder (carry-select)	~100K	0.001	Same structure as FPGA, smaller transistors
384-bit barrel shifter	~150K	0.001	9-stage mux network
384-bit comparator	~50K	<0.001	Cascaded with early termination
Cross-multiply control	~80K	<0.001	Sequencing for 2 multiplies + compare
Register file (16 × 384 V + 16 × 384 R)	~3M	0.025	12,288 bits in SRAM cells
Remainder SRAM (4 KB)	~1M	0.008	32 nodes × 128 bytes
Instruction fetch/decode	~200K	0.002	Simpler than float decode (no special cases)
Warp scheduling (shared across 32 QIUs)	~500K / 32	<0.001	Amortized per QIU
Per-QIU total	~7.1M	~0.058	

7.2 Per-SM Area

Component	Transistor Estimate	Area at 4nm (mm ²)
64 QIUs	454M	3.7
Warp schedulers (2)	1M	0.008
Local reduction unit	2M	0.016
Shared SRAM (256 KB)	50M	0.4

Component	Transistor Estimate	Area at 4nm (mm ²)
Instruction cache (64 KB)	12M	0.1
Interconnect and control	10M	0.08
Per-SM total	~529M	~4.3

7.3 Full Chip Area

Component	Count	Transistors (B)	Area (mm ²)
SMs	80	42.3	344
Global reduction network	1	0.5	4
L2 cache (96 MB)	1	19.2	155
HBM3 PHY + controllers	6	3.0	24
PCIe gen5 + NVLink	1	1.5	12
Host interface + scheduling	1	1.0	8
Power management	1	0.5	4
I/O ring and ESD	1	—	30
Total		~68B	~581

7.4 Comparison with Current GPUs

Property	H100 (4nm)	VDR-Q335 (4nm, projected)
Transistors	80B	68B
Die area	814 mm ²	581 mm ²
SMs	132	80
Compute units per SM	128 FP32	64 QIU (384-bit)
Total compute units	16,896 FP32	5,120 QIU
Register file total	~20 MB	~28 MB (wider registers)
L2 cache	50 MB	96 MB
HBM capacity	80 GB	96 GB
HBM bandwidth	3.35 TB/s	4.9 TB/s
Floating-point units	Yes (all precisions)	None
Special function units	Yes (sin, cos, exp, rsqrt)	None
Tensor cores	Yes	None (replaced by QIU multiply)
	(FP16/BF16/FP8/INT8)	
Q335 multiply throughput	~85M/s (integer ALU, from VDR-18 GA3)	~5.1T/s (QIU, projected)

The VDR-Q335 chip is smaller than H100 (581 mm² versus 814 mm²) because it contains no floating-point hardware, no tensor cores, no SFUs, and fewer SMs. It is simpler because every compute unit performs the same fixed-width integer operations — no mode switching between FP16/FP32/FP64/INT8, no special-case handling for denormals/NaN/infinity, no rounding mode configuration. The Q335 multiply throughput is approximately $60 \times$ higher than H100’s integer ALU performance for Q335 workloads because the QIUs are purpose-built for 384-bit arithmetic rather than repurposing 32-bit integer units.

8. Performance Projections

8.1 Per-Operation Throughput

Operation	Cycles	Time at 2 GHz	Throughput (5,120 QIUs)
Q335 addition	1	0.5 ns	10.2 T ops/sec
Q335 multiplication	2	1.0 ns	5.1 T ops/sec
Q335 divmod (SHR335)	0 (routing)	0 ns	∞ (simultaneous with multiply)
Q335 comparison	1	0.5 ns	10.2 T ops/sec
Fraction unification	3	1.5 ns	3.4 T ops/sec
Fact match (per fact)	1-2	0.5-1.0 ns	5.1-10.2 T matches/sec
REDUCE global sum	7	3.5 ns	286 M reductions/sec
Softmax surrogate (1K logits, full chip)	~20	10 ns	100 M softmax/sec

8.2 SRE Case Study Projection

The SRE investigation from VDR-18 (1 MB Prometheus JSON, 200 endpoints, 18,000 metric values) required 769 VDR tokens and 1.5 ms of primitive GPU time on the GPU mapping analysis. On the integer-native chip:

Phase	VDR Tokens	Primitive Operations	Projected Time
Data acquisition	72	1.2M (parse + KB store)	$\sim 0.2 \mu s$
Filtering + threshold	38	29K (comparisons)	$\sim 0.003 \mu s$

Phase	VDR Tokens	Primitive Operations	Projected Time
Deployment correla- tion	108	54K (cross-reference)	$\sim 0.005 \mu s$
Statistics + ranking	44	5K (aggregation)	$\sim 0.0005 \mu s$
Complex transform	92	75K (script-equivalent)	$\sim 0.007 \mu s$
Versioned project	183	20K (KB operations)	$\sim 0.002 \mu s$
Formatted output	232	86K (grammar + format)	$\sim 0.008 \mu s$
Total primitives	769	$\sim 1.5M$	$\sim 0.2 \mu s$

The primitive computation completes in approximately 200 nanoseconds. The bottleneck is entirely the LLM forward pass generating the 769 tokens, not the primitive operations. The primitives are not merely computationally invisible as VDR-18 stated for GPU — they are approximately $50,000 \times$ faster than generating a single LLM token.

8.3 Training Step Projection

For a 10-million parameter model (feasible for exact VDR training):

Operation	Parameters	Ops per Param	Total Ops	Time (5,120 QIUs)
Forward pass (matrix multiply)	10M	~ 100 muls	1B	~ 0.2 ms
Softmax (per layer)	$\sim 10K$ per layer $\times 12$ layers	~ 50 ops	6M	$\sim 0.6 \mu s$
Backward pass (gradient)	10M	~ 200 muls	2B	~ 0.4 ms
SGD parameter update	10M	1 mul + 1 sub	20M	$\sim 2 \mu s$
Constraint check	15 constraints	$\sim 1K$ ops each	15K	$\sim 1.5 \mu s$
Denominator budget sweep	10M	1 compare	10M	$\sim 1 \mu s$

Operation	Parameters	Ops per Param	Total Ops	Time (5,120 QIUs)
Total training step			~3B	~0.6 ms

A complete exact training step on a 10-million parameter model completes in approximately 0.6 milliseconds. This is roughly 1,600 training steps per second. For comparison, the Python prototype performs a single training step on a tiny model in seconds. The dedicated silicon is approximately $5,000 \times$ faster than the Python prototype and approximately $50 \times$ faster than the FPGA implementation for the same operations.

8.4 Memory Bandwidth Analysis

The performance ceiling for large workloads is memory bandwidth, not compute.

Workload	Operand	Result Writes	Total Bandwidth	Time at 4.9 TB/s
	Reads			
10M param SGD update	$10M \times 96 \text{ B}$ (param + grad)	$10M \times 48 \text{ B}$	1.44 GB	0.29 ms
10M param forward (one layer)	$10M \times 48 \text{ B}$	$10M \times 48 \text{ B}$	0.96 GB	0.20 ms
1M fact scan	$1M \times 48 \text{ B}$	Match indices	48 MB	0.010 ms
Attention QK^T (S=1024, H=64)	$2M \times 48 \text{ B}$	$1M \times 48 \text{ B}$	144 MB	0.029 ms

The SGD update is approximately balanced between compute (0.2 ms for multiplications) and memory (0.29 ms for operand transfer). The fact scan is heavily memory-bound — the per-fact match operation takes 1 cycle but loading the fact takes multiple cycles of memory latency. The attention computation is moderately memory-bound.

This is the expected profile. Modern GPU workloads are overwhelmingly memory-bandwidth-bound for regular parallel operations. The VDR-Q335 chip follows the same pattern — the compute is fast enough that the memory system is the bottleneck. The response is the same as for conventional GPUs: maximize memory bandwidth (HBM3 at 4.9 TB/s), minimize data movement (keep operands in L2 and shared memory), and

overlap computation with memory access (dual warp issuing to hide latency).

9. What This Chip Cannot Do

This section lists capabilities the chip does not provide, consistent with the project’s practice of stating limitations explicitly.

The chip cannot run conventional neural network inference. There are no floating-point units. A model trained in float16, bfloat16, or float32 cannot execute on this hardware. The chip runs only VDR-LLM-Prolog workloads with exact integer arithmetic. This is by design — the entire die area optimization depends on eliminating float hardware.

The chip cannot train production-scale language models. A 1-billion parameter model at 384 bits per parameter requires 48 GB for parameters alone, plus comparable storage for gradients and optimizer state. A 96 GB HBM capacity can accommodate this with tight memory management, but larger models require multi-device parallelism. The target scale for exact training is millions to tens of millions of parameters — sufficient for research into training dynamics and exact reproducibility, not for training GPT-class models.

The chip cannot accelerate functional remainder resolution. Newton iteration, Taylor series, and other convergent computations are inherently sequential — each step depends on the previous. The chip can accelerate the per-step arithmetic (each Newton step’s multiply is fast) but cannot parallelize the iteration itself. Functional remainder resolution is a host CPU responsibility that benefits from the chip only through faster individual arithmetic operations.

The chip cannot perform variable-precision arithmetic natively. The QIU is fixed at 384 bits. Values requiring wider precision (rare in practice but possible during deep remainder nesting) must be handled by multi-word arithmetic in software, using the QIU as a 384-bit building block. The Q335 frame provides 290 orders of magnitude of headroom over typical training denominators, so this limitation rarely manifests.

The chip cannot replace a general-purpose GPU for non-VDR workloads. It has no graphics pipeline, no video decode, no ray tracing, no conventional machine learning inference. It is a single-purpose accelerator for exact integer arithmetic in the VDR-LLM-Prolog system.

10. The Strategic Argument

A GPU manufacturer considering this design faces a market question: is the VDR-LLM-Prolog system valuable enough to justify dedicated silicon?

The technical answer is clear from the architecture. The chip is smaller than current GPUs (581 mm² versus 814 mm²), simpler (no float special cases, no mode switching, no rounding modes), and achieves 60 × higher throughput on target workloads. The

manufacturing cost is lower due to smaller die area. The power consumption is lower because integer arithmetic consumes less switching energy than float (no mantissa alignment, no exponent handling, no normalization) and because SHR335 divmod consumes zero switching energy.

The market answer depends on adoption. The VDR-LLM-Prolog system offers properties no float-based system can match: exact reproducibility, structural security with provably impossible data-access jailbreaking, complete provenance chains, flat per-turn cost scaling, and knowledge accumulation that makes every session cheaper than the last. Each property addresses a real market need: regulated industries need reproducibility, enterprises need security guarantees, compliance teams need audit trails, and cost-sensitive deployments need predictable economics.

The transition path does not require this chip as a first step. VDR-18 specified the GPU mapping using existing integer ALUs on conventional GPUs. The H100’s integer ALUs achieve approximately 85 million Q335 multiplications per second — sufficient for the system’s needs given 85-97% token reduction. A GPU manufacturer could support VDR workloads on existing hardware by exposing wider integer intrinsics, adding SHR335-equivalent instructions to the ISA, and optimizing the integer ALU pipeline for 384-bit operations. The dedicated chip represents the endpoint: what happens when the hardware is designed for the workload rather than the workload adapted to the hardware.

The FPGA at \$200 proves the architecture. Conventional GPUs with integer ALU optimization prove the scale. The dedicated chip proves the ceiling — and that ceiling is $60\times$ higher throughput at lower cost and lower power than repurposing float-era hardware for integer-era computation.

11. Fabrication and Packaging

11.1 Process Node

The design targets TSMC N4P (4nm-class) or equivalent. The transistor budget of approximately 68 billion and die area of approximately 581 mm^2 are well within the manufacturing capabilities demonstrated by current GPU products at this node. The design is simpler than current GPUs — no mixed-precision datapaths, no special function units, no complex rounding logic — which should yield higher transistor density in the compute units and better timing closure.

11.2 Packaging

The chip uses a 2.5D CoWoS-S (Chip-on-Wafer-on-Substrate) package with 6 HBM3 stacks, following the established packaging approach for high-bandwidth-memory GPU products. The interposer connects the compute die to the HBM stacks with short, high-bandwidth silicon traces. The package dimensions are comparable to current GPU products (approximately $100\text{mm} \times 70\text{mm}$).

11.3 Power Envelope

Component	Estimated Power (W)
80 SMs (5,120 QIUs at 2 GHz)	250
L2 cache (96 MB)	30
HBM3 PHY + controllers	60
Global reduction network	5
I/O (PCIe gen5 + NVLink)	25
Miscellaneous (clocking, power management)	30
Total TDP	~400 W

The 400W TDP is significantly below current high-end GPU TDPs (H100 at 700W, B200 at 1000W). The savings come from three sources: no floating-point units (float datapaths are power-hungry due to mantissa alignment and normalization logic), no SFUs (transcendental approximation circuits consume significant dynamic power), and no tensor cores (the largest single power consumer in current GPU SMs). Integer arithmetic at 384 bits consumes more power per operation than 32-bit integer arithmetic but less than float arithmetic at any precision because the datapath is simpler — carry propagation and integer multiplication are electrically simpler than floating-point addition (which requires exponent comparison, mantissa shift, and post-normalization).

11.4 Integer Arithmetic Power Advantage

The power savings from integer-versus-float deserves quantification. A 32-bit floating-point addition requires approximately 5 picojoules per operation on a 4nm process, accounting for exponent comparison (5 bits), mantissa alignment (up to 23-bit shift), mantissa addition (24 bits), normalization (leading-zero detection and shift), and rounding. A 384-bit integer addition requires approximately 3 picojoules per operation — less than a single float32 addition despite being $12 \times$ wider, because the operation is a single carry-chain propagation with no alignment, no normalization, and no rounding.

For multiplication: a float32 multiply requires approximately 15 picojoules (24-bit mantissa multiply plus exponent add plus normalization). A 384-bit integer multiply requires approximately 200 picojoules (full parallel multiplier tree). The integer multiply is more expensive per operation, but VDR performs 85-97% fewer operations per prompt. At 90% token reduction, the energy per prompt for multiplication is: $200 \text{ pJ} \times 0.1 = 20 \text{ pJ}$ effective, versus $15 \text{ pJ} \times 1.0 = 15 \text{ pJ}$ for float. Nearly equivalent per prompt, with the VDR version producing exact results.

12. Development Path from FPGA to ASIC

12.1 Phase Sequence

Phase	Duration	Deliverable	Validates
1. FPGA prototype (VDR-21)	32 weeks	10-core Q335 on Zynq-7020	Architecture, ISA, bit-identical results
2. FPGA scale-up	12 weeks	60-core Q335 on Zynq-7045 or UltraScale+	Scaling behavior, reduction network, memory bandwidth
3. RTL for ASIC	20 weeks	Synthesizable Verilog/SystemVerilog for Q335 core	Timing closure at 2 GHz target, area estimates
4. Full chip RTL	30 weeks	Complete chip design with memory controllers, I/O, power management	Physical design readiness
5. Tapeout	16 weeks	GDSII to foundry	Manufacturing
6. Bring-up and validation	12 weeks	Silicon validation against 884-test suite	Bit-identical results on silicon
Total	~122 weeks	Production silicon	

12.2 Risk Reduction Through FPGA

The FPGA phases (1 and 2) reduce ASIC risk in specific ways.

The ISA is validated on FPGA before committing to silicon. Any instruction that proves unnecessary, incorrectly specified, or insufficiently performant is modified in the FPGA bitstream at zero manufacturing cost. The ASIC ISA is frozen only after FPGA validation is complete.

The microprogram library is validated on FPGA. The six microprograms (fact_match, q335_add, q335_mul, dot_product, constraint_eval, softmax_surr) are tested at scale on the 60-core FPGA before being committed as the baseline software for the ASIC. Additional microprograms identified during FPGA testing are added before tapeout.

The memory hierarchy is characterized on FPGA. The DMA transfer patterns, double-buffering effectiveness, shared BRAM access patterns, and remainder BRAM utilization are measured on the FPGA under realistic workloads. These measurements inform the ASIC memory hierarchy sizing (L2 cache capacity, shared memory per SM, HBM configuration).

The Zig host interface is validated on FPGA. The runtime's ability to dispatch operations, manage results, and fall back to software is tested end-to-end on the FPGA before the ASIC exists. When silicon arrives, the Zig runtime requires only a driver change (FPGA AXI registers → ASIC PCIe BAR), not an architectural change.

13. Comparison with Alternative Approaches

13.1 Conventional GPU with Software Q335

Running VDR-LLM-Prolog on existing GPU hardware using 32-bit integer ALUs for Q335 arithmetic. This is the VDR-18 approach. It works — 85M Q335 muls/sec on H100 is sufficient for the workloads. But it repurposes hardware designed for float: the float units are idle, the tensor cores are idle, the SFUs are idle. The customer pays for silicon they don't use. The integer-native chip eliminates this waste — every transistor contributes to the workload.

13.2 Conventional GPU with Integer Tensor Core Adaptation

A conventional GPU vendor adds support for 384-bit integer operands in the tensor core datapath. This is a smaller change than a full integer-native chip — it modifies the tensor core but retains float units and SFUs for conventional workloads. The chip remains a general-purpose GPU that also accelerates VDR. The tradeoff: VDR performance is better than software Q335 on integer ALUs but worse than a dedicated integer-native chip, and the chip remains expensive because it contains float hardware. This is the likely transition step if VDR adoption grows but doesn't justify a dedicated product.

13.3 Custom ASIC with Mixed Workload Support

A chip that includes both 384-bit QIUs and conventional float units, supporting VDR and conventional ML workloads on the same device. This maximizes flexibility but reduces either QIU count (less VDR throughput) or float unit count (less conventional ML throughput) relative to dedicated designs. The die area overhead of supporting both datapaths reduces the efficiency advantage that motivates the integer-native design.

13.4 FPGA at Larger Scale

Deploying VDR-LLM-Prolog on large FPGAs (UltraScale+ at 200+ cores) without ever moving to ASIC. This avoids the cost and risk of silicon tapeout. The tradeoff: FPGA clock speed (150-300 MHz) is $7\text{-}13 \times$ lower than ASIC (2 GHz), and FPGA logic density is approximately $10 \times$ lower than ASIC at the same node. For power-constrained edge deployments or low-volume applications, FPGA may be the right choice. For datacenter deployment where throughput per watt matters, ASIC is the better investment.

Approach	Q335 Muls/sec	Die Cost	Float Support	VDR Optimized
GPU + software Q335	85M	High (paying for unused float)	Full	No
GPU + integer tensor core	~500M	High	Full	Partial

Approach	Q335 Muls/sec	Die Cost	Float Support	VDR Optimized
Mixed ASIC	~2T	Very high	Reduced	Partial
FPGA (200 cores)	~3.3B	Low	None	Yes (but slow clock)
Integer-native ASIC (this paper)	~5.1T	Moderate	None	Yes

Appendix A: QIU Transistor Budget Detail

A.1 384-bit Parallel Multiplier Breakdown

Sub-component	Description	Transistors
6 × 6 array of 64-bit multipliers	36 Booth-encoded multipliers	1,080K
Partial product reduction (Wallace tree)	35 levels of carry-save adders	840K
Final carry-propagate adder (768-bit)	Carry-select, 12 × 64-bit blocks	150K
Pipeline registers (2 stages)	768-bit × 2	92K
Control logic	Sequencing, stall, output routing	40K
SHR335 routing	Zero transistors — physical wire routing	0
Total multiplier		~2,202K

The 384×384 multiplier dominates the QIU transistor budget at approximately 2.2M transistors, consistent with the general rule that multiplier area scales as $O(n^2)$ with operand width. The Booth encoding reduces the number of partial products by approximately half compared to a naive array multiplier. The Wallace tree reduction converts the partial product matrix to two rows (carry and sum) in $O(\log n)$ levels of carry-save adders, which are then combined by a final carry-propagate adder.

The SHR335 routing is explicitly listed at zero transistors to emphasize the hardware benefit of the fixed-denominator design.

A.2 Register File Implementation

Component	Description	Transistors
16 × 384-bit V registers	6,144 bits × 6T SRAM cells	36,864 cells = 221K
16 × 384-bit R registers	6,144 bits × 6T SRAM cells	221K
4 × 32-bit index registers	128 bits × 6T	768 cells = 5K
Read ports (4 per bank)	Read decoders + sense amps	80K
Write ports (2 per bank)	Write decoders + drivers	40K
Total register file		~567K

A.3 Remainder SRAM

Component	Description	Transistors
32 nodes × 128 bytes = 4 KB	32,768 bits × 6T SRAM cells	196,608 cells = ~1,180K
Address decoder	5-bit node index	10K
Read/write control	Single-port SRAM controller	15K
Total remainder SRAM		~1,205K

Appendix B: Memory Bandwidth Saturation Analysis

B.1 Bandwidth Required by Operation Type

Operation	Reads (B/op)	Writes (B/op)	Ops/sec at 5.1T muls/sec	Bandwidth (TB/s)
Q335 add	96 (2 × 48)	48	10.2T	1,469 (read-bound)
Q335 mul	96	96 (V+R)	5.1T	979
Fact match	48 (1 fact)	4 (index)	10.2T	530
Dot product (H=64)	6,144 (64 × 48 × 2)	48	167K per QIU	1.03
SGD update	144 (param+grad+lr)	48	5.1T	979

B.2 Achievable Throughput Under Bandwidth Constraint

Operation	Compute Time	Memory Time at 4.9 TB/s	Bottleneck	Achievable Throughput
Q335 add (bulk, no reuse)	0.5 ns / 10.2T/s	29 ps / 33.5T/s	Compute	10.2T/s
Q335 mul (bulk, no reuse)	1.0 ns / 5.1T/s	39 ps / 25.0T/s	Compute	5.1T/s
Fact match (1M facts, cold)	—	9.8 μ s for 48 MB	Memory	~102K queries/s
Fact match (1M facts, L2 cached)	—	~1 μ s	Compute	~1M queries/s
SGD update (10M params, streaming)	2 μ s	294 μ s	Memory	34K steps/s
Matrix multiply (1K $\times$ 1K)	0.5 ms	0.2 ms	Compute	2K matmuls/s

Bulk arithmetic on data already in registers or shared memory is compute-bound. Streaming operations on HBM-resident data are memory-bound. Cached operations (KB facts that fit in L2) fall between. This matches the profile of current GPUs and validates the memory hierarchy design: L2 cache sized to hold the working set, shared memory for per-SM reuse, registers for per-QIU operands.

Appendix C: Power Comparison Across Platforms

C.1 Energy per Q335 Operation

Platform	Q335 Add Energy	Q335 Mul Energy	Q335 Divmod Energy	Source
Python (Fraction)	$\sim 50 \mu\text{J}$	$\sim 500 \mu\text{J}$	$\sim 100 \mu\text{J}$	Measured: CPython inter- preter + arbitrary preci- sion
Zig (host CPU, 65W)	$\sim 1.3 \text{ nJ}$	$\sim 13 \text{ nJ}$	$\sim 1 \text{ nJ}$	Estimated: 3 GHz, 384-bit software
FPGA (Zynq, 1.9W)	$\sim 13 \text{ pJ}$	$\sim 114 \text{ pJ}$	$\sim 13 \text{ pJ}$	Estimated: 150 MHz, per-core power
GPU H100 (700W)	$\sim 8 \text{ pJ}$	$\sim 82 \text{ pJ}$	$\sim 8 \text{ pJ}$	Estimated: integer ALU, 1.8 GHz
VDR- Q335 ASIC (400W)	$\sim 3 \text{ pJ}$	$\sim 39 \text{ pJ}$	0 pJ	Projected: 2 GHz, pure integer

C.2 Energy per Complete SRE Investigation

Platform	Tokens	Energy per Token	Total Energy	Wall- Clock	Cost at \$0.10/kWh
Conventional LLM (H100)	25,100	$\sim 280 \text{ mJ}$	$\sim 7.0 \text{ kJ}$	$\sim 660 \text{ s}$	$\sim \$0.00019$
VDR on H100 (VDR- 18)	769	$\sim 29 \text{ mJ}$	$\sim 22 \text{ J}$	$\sim 9 \text{ s}$	$\sim \$0.0000006$
VDR on FPGA	769	$\sim 2.5 \text{ mJ}$	$\sim 1.9 \text{ J}$	$\sim 12 \text{ s}$	$\sim \$0.00000005$
VDR on ASIC (this paper)	769	$\sim 520 \mu\text{J}$	$\sim 0.4 \text{ J}$	$\sim 0.5 \text{ s}$	$\sim \$0.00000001$

The ASIC completes the SRE investigation in approximately 0.5 seconds using 0.4 joules of energy. The conventional LLM approach uses 7,000 joules — $17,500 \times$ more energy for the same investigation, with 25% data coverage versus 100% and arithmetic errors versus exact results.

C.3 Operations per Watt

Platform	Q335 Adds/Watt	Q335 Muls/Watt	Fact Matches/Watt
Python	20	2	0.2
Zig CPU	770K	77K	15K
FPGA (1.9W)	79M	8.8M	4.8M
GPU H100 (700W)	122M	12M	7M
VDR-Q335 ASIC (400W)	25.5B	12.8B	6.4B

The ASIC achieves approximately $200 \times$ better operations-per-watt than the FPGA for multiplications, and approximately $1,000 \times$ better than GPU H100. The combination of higher clock speed ($13 \times$), more cores ($512 \times$), and lower total power ($0.57 \times$) produces the multiplicative improvement. For deployments where energy cost or thermal budget is a constraint — datacenter at scale, edge devices, mobile — the efficiency advantage is decisive.

Appendix D: Instruction Latency Comparison Across Implementations

Instruction	Python	Zig CPU (est.)	FPGA (150 MHz)	ASIC (2 GHz)	ASIC vs Python
Q335 add	5,000 ns	20 ns	6.7 ns	0.5 ns	$10,000 \times$
Q335 mul	50,000 ns	200 ns	60 ns	1.0 ns	$50,000 \times$
Q335 divmod	10,000 ns	15 ns	6.7 ns (0 logic)	0 ns (routing)	∞
Fraction unify	100,000 ns	420 ns	127 ns	1.5 ns	$66,667 \times$
Fact query (200 facts)	500,000 ns	4,000 ns	1,100 ns	10 ns	$50,000 \times$
Softmax 100 logits	2,500,000 ns	25,000 ns	3,300 ns	10 ns	$250,000 \times$
Dot product H=64	3,200,000 ns	13,000 ns	5,970 ns	32 ns	$100,000 \times$

The Python column represents the reference implementation that validated the VDR arithmetic across 884 tests. The ASIC column represents the projected performance of the dedicated silicon. The ratio between them — $10,000 \times$ to $250,000 \times$ — reflects the transition from interpreted arbitrary-precision arithmetic on a general-purpose CPU to fixed-width integer arithmetic on purpose-built silicon. The Zig column represents the intermediate implementation that the host CPU runs for control-plane operations and software fallback.

Appendix E: Comparison with Existing Integer Accelerators

E.1 Existing Integer Hardware in AI Context

Accelerator	Integer Width	Purpose	Q335 Applicability
Google TPU v4 INT8	8-bit	Quantized inference	None — too narrow by $48 \times$
NVIDIA INT8 tensor core	8-bit	Quantized inference	None — too narrow, float accumulation
NVIDIA INT4 tensor core	4-bit	Ultra-quantized inference	None — too narrow by $96 \times$
Cerebras CS-2 INT16	16-bit	Sparse linear algebra	None — too narrow by $24 \times$
Graphcore MK2 INT32	32-bit	Mixed precision	Partial — could do multi-word Q335, very slow
Intel AMX INT8	8-bit	x86 matrix extension	None — too narrow, CPU-bound
Apple Neural Engine INT8	8-bit	Mobile inference	None — too narrow

No existing accelerator provides native support for 384-bit integer arithmetic. All existing integer hardware targets low-precision quantized inference (4-8 bit) — the opposite end of the precision spectrum from VDR’s 384-bit exact arithmetic. The VDR-Q335 ASIC would be the first accelerator designed for exact wide-integer arithmetic in an AI context.

E.2 Existing Wide-Integer Hardware Outside AI

System	Integer Width	Purpose	Q335 Applicability
Cryptographic accelerators (RSA, ECC)	256-4096 bit	Modular exponentiation	Partial — similar width, different operation mix (modular reduction vs Q335 divmod)
FPGA-based BigInteger	Variable	Arbitrary precision	Partial — general but not optimized for fixed-frame Q335
ZKP accelerators (Ingonyama, Cysic)	256-384 bit	Zero-knowledge proof field arithmetic	Closest match — similar width, similar multiply-heavy workload, but modular reduction not power-of-two shift

Zero-knowledge proof accelerators are the closest existing precedent. They perform 256-384 bit integer multiplication with modular reduction over a prime field. VDR-Q335 performs 384-bit integer multiplication with power-of-two reduction (bit extraction). The QIU design could potentially support ZKP workloads by adding a modular reduction instruction alongside SHR335, making the chip applicable to both exact AI arithmetic and zero-knowledge cryptography — two distinct markets that share a hardware substrate.

Appendix F: Thermal and Physical Design

F.1 Thermal Dissipation

Parameter	Value	Notes
TDP	400 W	Section 11.3 estimate
Junction temperature (max)	95°C	Standard for GPU-class devices
Ambient (datacenter)	35°C	Standard datacenter operating temperature
Thermal resistance (junction to heatsink)	0.05°C/W	CoWoS package with vapor chamber
Required heatsink capacity	400 W at $\Delta T=60^\circ\text{C}$	Standard GPU cooling solution sufficient
Cooling solution	Vapor chamber + dual-fan	Same as current GPU reference designs

The 400W TDP is well within the cooling capabilities of standard datacenter GPU cooling solutions. Current GPUs at 700-1000W use the same cooling approach with larger heatsinks. The VDR-Q335 chip's lower TDP means it can use smaller, quieter cooling,

or operate with greater thermal headroom in the same cooling envelope.

F.2 Board Design

Parameter	Value	Notes
Form factor	PCIe gen5 x16, dual-slot	Standard GPU form factor
Board power	450 W (chip TDP + VRM losses + HBM)	Single 16-pin power connector
Memory	6 × HBM3 stacks, 96 GB total	CoWoS-S interposer
Host interface	PCIe gen5 x16 (64 GB/s bidirectional)	Standard
Inter-chip link	NVLink equivalent (optional)	For multi-chip configurations
Board dimensions	267mm × 111mm	Standard dual-slot GPU

Appendix G: Multi-Chip Configurations

G.1 Scaling for Larger Models

Configuration	QIUs	HBM Capacity	HBM Bandwidth	Q335 Muls/sec	Max Model (training)	Max Model (inference)
1 chip	5,120	96 GB	4.9 TB/s	5.1T	~10M params	~200M params
2 chips	10,240	192 GB	9.8 TB/s	10.2T	~25M params	~500M params
4 chips	20,480	384 GB	19.6 TB/s	20.5T	~60M params	~1B params
8 chips	40,960	768 GB	39.2 TB/s	41.0T	~120M params	~2B params

Model size limits account for parameter storage, gradient storage, optimizer state (for training), attention matrices, KB fact storage, and working buffers. Inference requires only parameters and attention matrices, enabling approximately $20 \times$ larger models than training on the same hardware.

G.2 Inter-Chip Communication

Pattern	Data Volume	Latency Requirement	Mechanism
Global reduction across chips	48 B per chip (one Q335 value)	$<1 \mu s$	NVLink ring reduce
Parameter synchronization	Full parameter set	Per training step	NVLink bulk transfer
KB fact partition distribution	Partition size	At initialization	NVLink or PCIe
Attention KV cache sharing	Per-layer KV vectors	Per token	NVLink

The inter-chip communication patterns mirror conventional model-parallel GPU training. The global reduction across chips extends the on-chip reduction network — the on-chip tree produces per-chip sums in 7 cycles, the inter-chip ring reduce produces the global sum in $O(\text{chips})$ steps. For 8 chips, the full reduction completes in approximately 3.5 ns (on-chip) + 100 ns (inter-chip NVLink) = ~ 104 ns. This is fast enough that global operations (softmax denominator, gradient norm) do not bottleneck the system.

References

- [[HOWL-VDR-1-2026](#)] VDR Arithmetic: Value, Denominator, Remainder. Exact Finite Arithmetic in Irreducible Triple Form. DOI: 10.5281/zenodo.15302702
- [[HOWL-VDR-4-2026](#)] Exact-Fraction Language Model Architecture. From Arithmetic Library to Working Transformer in 24 Modules.
- [[HOWL-VDR-13-2026](#)] VDR in Physical Computation. Exact Arithmetic Where It Matters.
- [[HOWL-VDR-14-2026](#)] VDR-LLM-Prolog. A Complete System Specification for Exact Arithmetic Language Models with Structural Provenance. DOI: 10.5281/zenodo.20232194
- [[HOWL-VDR-18-2026](#)] VDR-LLM-Prolog: Performance. Integer Arithmetic on GPU Hardware.
- [[HOWL-VDR-21-2026](#)] VDR-LLM-Prolog on FPGA. Exact Integer Arithmetic in Custom Silicon: A 10-Core Q335 Processor on Zynq-7020.

[[HOWL-VDR-1-2026](#)] VDR Arithmetic: Value, Decimal, Remainder. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-1-2026>

[[HOWL-VDR-2-2026](#)] VDR Gym: Exact Arithmetic Across Fifteen Domains. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-2-2026>

[[HOWL-MATH-3-2026](#)] The Transcendental Hierarchy. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH-3-2026>

[[HOWL-MATH-4-2026](#)] The Universal Power-of-Two Basis. Github: <https://github.com/ghowland/papers/tree/main/papers/MATH-4-2026>

[[HOWL-VDR-14-2026](#)] VDR-LLM-Prolog. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-14-2026>

[[HOWL-VDR-21-2026](#)] VDR-LLM-Prolog on FPGA. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR-21-2026>

[[HOWL-VDR-4-2026](#)] Exact-Fraction Language Model Architecture. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR/HOWL-VDR-4-2026>

[[HOWL-VDR-13-2026](#)] VDR in Physical Computation. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR-13-2026>

[[HOWL-VDR-18-2026](#)] VDR-LLM-Prolog: Performance. Github: <https://github.com/ghowland/papers/tree/main/papers/VDR-18-2026>